



**"RANKED 45
IN INDIA**

#University Category



**WORLD
UNIVERSITY
RANKINGS**

**NO.1 PVT. UNIVERSITY IN
ACADEMIC REPUTATION IN INDIA**



**ACCREDITED GRADE 'A'
BY NAAC**



**PERFECT SCORE OF 150/150 AS A TESTAMENT
TO EXCEPTIONAL E-LEARNING METHODS**

1



Applied Machine Learning

Unit 05 – Lecture 11: Multiple Linear Regression

Tofik Ali

School of Computer Science, UPES Dehradun

Autumn 2026

Building, deriving, interpreting and validating a model with many predictors

Topic focus: **a coefficient is a conditional statement, not a fact about a column**

Repository: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: James et al., *An Introduction to Statistical Learning with Python*, Ch. 3–6.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 3.; Deisenroth, Faisal and Ong, *Mathematics for Machine Learning*, Ch. 9.; Müller and Guido, *Introduction to Machine Learning with Python*, §2.3.

Model
oo

Derivation
ooooooo

By hand
ooooooo

Meaning
oooooooooooo

VIF
ooooooooo

Missing
oooo

Checks
ooooooo

Summary
oo

Quick Links

The model

The derivation

By hand

Interpretation

Standardized

Collinearity

Missing data

Validation

Summary

Agenda

The model and the design matrix

Deriving $\hat{\beta} = (X^T X)^{-1} X^T y$

The five flats, worked completely

What a coefficient means

Standardized (beta) coefficients

Multicollinearity

Missing data in a regression

Model validation

Summary

How this lecture works

Every idea appears twice

1. **By hand** — five flats, two predictors, a 3×3 matrix inverted with cofactors. Integer arithmetic throughout; nothing rounds.
2. **In code** — the same quantities on 442 diabetes patients, so you see that `LinearRegression` solves exactly that system and nothing more.

Commit before you look

Four slides ask you to predict a number before it is revealed. Write your guess down. Being wrong on paper is how the idea sticks.

The centrepiece is a **predictor whose coefficient is +0.47 on its own and −1.09 once its neighbours join the model**. Everything after it exists to explain why that is not a bug.

Where we are

You already have the one-predictor case

$\hat{y} = b_0 + b_1x$, least squares, the two normal equations, R^2 , and maximum likelihood under Gaussian noise. Nothing in this lecture replaces any of it.

Real tables have many columns

A patient has ten measurements, a flat has five attributes, a loan application has forty. Fitting them one at a time answers the wrong question.

Today's five questions

How do we write the model for p predictors? Where does $\hat{\beta} = (X^T X)^{-1} X^T y$ come from, and when does it fail? What does a coefficient *mean*? Which coefficients can be compared? And how do we know the model is any good?

One sentence to carry throughout

A multiple-regression coefficient is the effect of a predictor **holding the others fixed**. Change the others and you change the coefficient.

One seed — and one thing we deliberately do not fix

Seed 0, for everything arbitrary

Splits, the eight-patient subset, the simulated missing cells, the imputer, the junk columns, the collinear pair. The data itself (`load_diabetes(scaled=False)`, `load_wine`) is bundled and never generated.

The bootstrap is a *sweep*

2000 resamples, `range(0, 2000)`. **The spread across them is the whole result** — it is how we show a coefficient is not reportable. Collapsing it to one seed would delete the demonstration.

The distinction is worth learning

A seed you picked arbitrarily should be fixed and stated. A set of seeds you are *deliberately varying* is an experiment, and its variability is data. Every number in this deck reproduces exactly.

From one predictor to p

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} + \varepsilon_i, \quad i = 1, \dots, n$$

Stack the rows and it is one equation

$$\underbrace{y}_{n \times 1} = \underbrace{X}_{n \times (p+1)} \underbrace{\beta}_{(p+1) \times 1} + \varepsilon,$$

with a leading column of ones in X so that β_0 is an ordinary coefficient and needs no special treatment.

What “linear” means

Linear **in the parameters**, not in the predictors. $y = \beta_0 + \beta_1 x + \beta_2 x^2$ is a multiple linear regression with $x_2 = x^2$. $y = \beta_0 x^{\beta_1}$ is not.

The geometry

One predictor fits a *line* in the plane. Two fit a *plane* in three dimensions. p fit a p -dimensional **hyperplane**.

By hand: five flats, two predictors

Price in lakh of rupees. x_1 is the number of bedrooms; x_2 is the floor area in units of 50 m^2 .

flat	A	B	C	D	E
bedrooms x_1	1	2	3	4	5
area x_2	2	3	4	3	4
price y	8	13	15	11	12

$$X = \begin{pmatrix} 1 & 1 & 2 \\ 1 & 2 & 3 \\ 1 & 3 & 4 \\ 1 & 4 & 3 \\ 1 & 5 & 4 \end{pmatrix}, \quad y = \begin{pmatrix} 8 \\ 13 \\ 15 \\ 11 \\ 12 \end{pmatrix}, \quad n = 5, \quad p = 2.$$

Look at flats B and D before we compute anything

Both are 150 m^2 . B has 2 bedrooms and costs 13; D has 4 and costs 11. **At a fixed area, an extra bedroom is worth -1 lakh.** Hold that thought.

The objective, written once

Least squares chooses b to minimise the sum of squared residuals:

$$S(b) = \sum_{i=1}^n (y_i - x_i^\top b)^2 = (y - Xb)^\top (y - Xb).$$

Expand, using $b^\top X^\top y = y^\top X b$ (both are 1×1 , so each is its own transpose):

$$S(b) = y^\top y - 2b^\top X^\top y + b^\top X^\top X b.$$

Three terms, three shapes

$y^\top y$ is a constant. $-2b^\top X^\top y$ is **linear** in b . $b^\top X^\top X b$ is a **quadratic form**, and $X^\top X$ is symmetric and positive semi-definite — which is exactly why this problem has a minimum and not a saddle.

Two facts of matrix calculus, and that is all you need

Linear term

$$\frac{\partial}{\partial b}(b^\top c) = c$$

Check componentwise: $b^\top c = \sum_j b_j c_j$, so
 $\partial/\partial b_k = c_k$.

Quadratic form, A symmetric

$$\frac{\partial}{\partial b}(b^\top A b) = 2A b$$

Check componentwise:
 $b^\top A b = \sum_j \sum_k b_j A_{jk} b_k$, so
 $\partial/\partial b_m = 2 \sum_k A_{mk} b_k$.

Apply both to $S(b) = y^\top y - 2b^\top X^\top y + b^\top X^\top X b$:

$$\nabla S(b) = -2X^\top y + 2X^\top X b.$$

And the second derivative

$\nabla^2 S(b) = 2X^\top X \succeq 0$, so every stationary point is a **global** minimum. There are no local minima to worry about — which is why regression is not an optimisation problem in the way that training a network is.

Set the gradient to zero: the normal equations

$$-2 X^T y + 2 X^T X \hat{\beta} = 0 \quad \Longleftrightarrow \quad \boxed{X^T X \hat{\beta} = X^T y}$$

and if $X^T X$ is invertible,

$$\boxed{\hat{\beta} = (X^T X)^{-1} X^T y}$$

This is the simple case, generalised

With one predictor, $X^T X = \begin{pmatrix} n & \sum x \\ \sum x & \sum x^2 \end{pmatrix}$ and $X^T y = \begin{pmatrix} \sum y \\ \sum xy \end{pmatrix}$. Solving the 2×2 system gives $b_1 = S_{xy}/S_{xx}$ and $b_0 = \bar{y} - b_1 \bar{x}$. **Same equations.**

Read the boxed line as a warning

The formula is only a formula *if* $X^T X$ can be inverted. The whole of the next slide is about when it cannot — and multicollinearity, later in the lecture, is the case where it *barely* can.

When is $X^T X$ invertible?

Proof of the equivalence, in three lines

Suppose $X^T X v = 0$ for some $v \neq 0$. Then $v^T X^T X v = \|X v\|^2 = 0$, so $X v = 0$: the columns of X are linearly dependent. Conversely if $X v = 0$ then $X^T X v = 0$. Hence

$$\text{null}(X^T X) = \text{null}(X), \quad \text{so} \quad \text{rank}(X^T X) = \text{rank}(X).$$

$X^T X$ is $(p+1) \times (p+1)$, so it is invertible exactly when $\text{rank}(X) = p + 1$: **full column rank**.

Failure 1: perfect collinearity

One column is an exact linear combination of the others. Duplicated columns, a total and its parts, all k dummies plus an intercept, the same quantity in two units.

Failure 2: $p + 1 > n$

More parameters than rows. Then $\text{rank}(X) \leq n < p + 1$ **whatever the numbers are**. Gene expression, text, any wide table.

The same condition, seen geometrically

What the normal equations say

Rearrange $X^T X \hat{\beta} = X^T y$:

$$X^T (y - X \hat{\beta}) = X^T e = 0.$$

Every column of X is orthogonal to the residual vector. The fitted values $\hat{y} = X \hat{\beta}$ are the orthogonal projection of y onto the column space of X , and e is what is left over.

The hat matrix

$\hat{y} = Hy$ with $H = X(X^T X)^{-1} X^T$. H is symmetric, $H^2 = H$, and $\text{tr}(H) = p + 1$ — the

Two consequences you can check on paper

Because the first column of X is all ones, $X^T e = 0$ forces $\sum_i e_i = 0$: **the residuals always sum to zero**. And because the j th column is x_j , it forces $\sum_i x_{ij} e_i = 0$: **the residuals are uncorrelated with every predictor, by construction**.

So do not be impressed

“My residuals have mean zero and no correlation with the inputs” is not evidence that the model is right. It is arithmetic.

Failure 1 in code: an exact linear combination

Add a third column $x_3 = x_1 + x_2$ to the five flats. Nothing is wrong with the data; the *matrix* is now rank deficient.

```
shape (5, 4) rank 3 (needs 4)
det(X'X) = 0.000000e+00
np.linalg.inv -> LinAlgError: Singular matrix
condition number of X'X: 7.692e+18
```

Failure 1 in code: an exact linear combination

Add a third column $x_3 = x_1 + x_2$ to the five flats. Nothing is wrong with the data; the *matrix* is now rank deficient.

```
shape (5, 4) rank 3 (needs 4)
det(X'X) = 0.000000e+00
np.linalg.inv -> LinAlgError: Singular matrix
condition number of X'X: 7.692e+18

LinearRegression still answers: [ 2. -2.  3.  1.]
a DIFFERENT coefficient vector: [ 2. -1.  4.  0.]
max |difference in predictions| 0.00e+00
```

Failure 1 in code: an exact linear combination

Add a third column $x_3 = x_1 + x_2$ to the five flats. Nothing is wrong with the data; the *matrix* is now rank deficient.

```
shape (5, 4) rank 3 (needs 4)
det(X'X) = 0.000000e+00
np.linalg.inv -> LinAlgError: Singular matrix
condition number of X'X: 7.692e+18

LinearRegression still answers: [ 2. -2.  3.  1.]
a DIFFERENT coefficient vector: [ 2. -1.  4.  0.]
max |difference in predictions| 0.00e+00
```

The library did not raise, and its answer is not unique

LinearRegression calls `lstsq`, which returns the *minimum-norm* solution instead of failing. Add c to b_1 and b_2 and subtract c from b_3 and the predictions do not change by 10^{-16} . **The predictions are fine; the coefficients mean nothing.** This is the dummy variable trap of Lecture 6 in its general form.

Failure 2 in code: more parameters than patients

Take 8 diabetes patients and all 10 features.

```
8 patients, 10 features + intercept -> X is (8, 11)
rank(X) = 8, so rank(X'X) = 8 but X'X is 11 x 11
np.linalg.inv -> LinAlgError: Singular matrix
lstsq minimum-norm solution ||b|| = 31.5783, SSE = 3.709e-24
add 100 * a null-space direction: ||b|| = 104.8675, SSE = 2.614e-23
```

Failure 2 in code: more parameters than patients

Take 8 diabetes patients and all 10 features.

```
8 patients, 10 features + intercept -> X is (8, 11)
rank(X) = 8, so rank(X'X) = 8 but X'X is 11 x 11
np.linalg.inv -> LinAlgError: Singular matrix
lstsq minimum-norm solution ||b|| = 31.5783, SSE = 3.709e-24
add 100 * a null-space direction: ||b|| = 104.8675, SSE = 2.614e-23
```

Both vectors fit all eight rows **exactly**, and they are two of an infinite family: the null space of X has dimension $11 - 8 = 3$, so you may move $\hat{\beta}$ anywhere in a 3-dimensional affine set at zero cost.

Failure 2 in code: more parameters than patients

Take 8 diabetes patients and all 10 features.

```
8 patients, 10 features + intercept -> X is (8, 11)
rank(X) = 8, so rank(X'X) = 8 but X'X is 11 x 11
np.linalg.inv -> LinAlgError: Singular matrix
lstsq minimum-norm solution ||b|| = 31.5783, SSE = 3.709e-24
add 100 * a null-space direction: ||b|| = 104.8675, SSE = 2.614e-23
```

Both vectors fit all eight rows **exactly**, and they are two of an infinite family: the null space of X has dimension $11 - 8 = 3$, so you may move $\hat{\beta}$ anywhere in a 3-dimensional affine set at zero cost.

This is the problem regularisation exists to solve

Ridge replaces $X^T X$ by $X^T X + \lambda I$, which is *always* invertible for $\lambda > 0$, and picks one member of that family. That is the next lecture. **Note what the disease is: not “too little signal” but “no unique answer”.**

Step 1: form $X^T X$ and $X^T y$

Every entry is a sum you can do in your head.

$$X^T X = \begin{pmatrix} n & \sum x_1 & \sum x_2 \\ \sum x_1 & \sum x_1^2 & \sum x_1 x_2 \\ \sum x_2 & \sum x_1 x_2 & \sum x_2^2 \end{pmatrix} = \begin{pmatrix} 5 & 15 & 16 \\ 15 & 55 & 52 \\ 16 & 52 & 54 \end{pmatrix}, \quad X^T y = \begin{pmatrix} 59 \\ 183 \\ 196 \end{pmatrix}.$$

$\sum x_1 = 1+2+3+4+5$	$= 15$	$\sum y = 8+13+15+11+12 = 59$
$\sum x_1^2 = 1+4+9+16+25$	$= 55$	$\sum x_1 y = 8+26+45+44+60 = 183$
$\sum x_1 x_2 = 2+6+12+12+20$	$= 52$	$\sum x_2 y = 16+39+60+33+48 = 196$

$X^T X$ is symmetric, and that is not a coincidence

$(X^T X)^T = X^T X$ for any X at all. So you only ever compute the upper triangle: six numbers here, not nine.

Step 2: invert it with cofactors

For the symmetric matrix $M = X^T X$, the cofactors along the first row are

$$A_{11} = 55 \cdot 54 - 52^2 = 266, \quad A_{12} = -(15 \cdot 54 - 52 \cdot 16) = 22, \quad A_{13} = 15 \cdot 52 - 55 \cdot 16 = -100,$$

$$\det M = 5(266) + 15(22) + 16(-100) = 1330 + 330 - 1600 = \mathbf{60}.$$

The remaining cofactors are $A_{22} = 5 \cdot 54 - 16^2 = 14$, $A_{23} = -(5 \cdot 52 - 15 \cdot 16) = -20$, $A_{33} = 5 \cdot 55 - 15^2 = 50$, so

$$(X^T X)^{-1} = \frac{1}{60} \begin{pmatrix} 266 & 22 & -100 \\ 22 & 14 & -20 \\ -100 & -20 & 50 \end{pmatrix}.$$

det = 60 is small relative to the entries

That is the first hint of what is coming: x_1 and x_2 correlate at +0.756, so the matrix is closer to singular than its size suggests. Perfectly invertible here — *barely* invertible is the subject of the multicollinearity section.

Step 3: multiply, and read the answer

$$\hat{\beta} = (X^T X)^{-1} X^T y = \frac{1}{60} \begin{pmatrix} 266 & 22 & -100 \\ 22 & 14 & -20 \\ -100 & -20 & 50 \end{pmatrix} \begin{pmatrix} 59 \\ 183 \\ 196 \end{pmatrix}$$

$266(59) + 22(183) - 100(196) = 15694 + 4026 - 19600 = 120$	$\Rightarrow b_0 = 120/60 = 2$
$22(59) + 14(183) - 20(196) = 1298 + 2562 - 3920 = -60$	$\Rightarrow b_1 = -60/60 = -1$
$-100(59) - 20(183) + 50(196) = -5900 - 3660 + 9800 = 240$	$\Rightarrow b_2 = 240/60 = 4$

$$\hat{y} = 2 - 1 \cdot x_1 + 4 \cdot x_2$$

The bedroom coefficient is negative. An extra 50 m² is worth +4 lakh; an extra bedroom *in the same area* is worth -1 lakh — which is exactly what flats B and D said before we started.

Step 4: check it, the way the derivation tells you to

	A	B	C	D	E
$\hat{y} = 2 - x_1 + 4x_2$	9	12	15	10	13
y	8	13	15	11	12
residual e	-1	+1	0	+1	-1

$$\sum e_i = 0, \quad \sum x_{1i}e_i = -1 + 2 + 0 + 4 - 5 = 0, \quad \sum x_{2i}e_i = -2 + 3 + 0 + 3 - 4 = 0.$$

Fit statistics

$$\text{SSE} = 1 + 1 + 0 + 1 + 1 = 4.$$

$$\bar{y} = 11.8, \text{ so } \text{SST} = 26.8.$$

$$R^2 = 1 - 4/26.8 = \mathbf{0.8507}.$$

$$R^2_{\text{adj}} = 1 - \frac{4/2}{26.8/4} = \mathbf{0.7015}.$$

$X^T e = 0$ is the check that matters

Those three zeros are the normal equations, read backwards. If your hand arithmetic gets $\hat{\beta}$ wrong, they will not come out — so compute them every time. Two minutes of insurance.

The same thing in code

```
X = np.c_[np.ones(5), x1, x2]
beta = np.linalg.inv(X.T @ X) @ (X.T @ y) # by hand
lr = LinearRegression().fit(np.c_[x1, x2], y) # by library
```

```
X'X =
[[ 5 15 16]
 [15 55 52]
 [16 52 54]]
X'y = [ 59 183 196]
det(X'X) = 60
60 * inv(X'X) =
[[ 266.   22. -100.]
 [  22.   14.  -20.]
 [-100.  -20.   50.]]
beta = inv(X'X) X'y = [ 2. -1.  4.]
LinearRegression intercept 2.000000000000 coef [-1.  4.]
max |difference| 8.62e-14
```

The same thing in code

```
X = np.c_[np.ones(5), x1, x2]
beta = np.linalg.inv(X.T @ X) @ (X.T @ y) # by hand
lr = LinearRegression().fit(np.c_[x1, x2], y) # by library
```

```
X'X =
[[ 5 15 16]
 [15 55 52]
 [16 52 54]]
X'y = [ 59 183 196]
det(X'X) = 60
60 * inv(X'X) =
[[ 266.   22. -100.]
 [  22.   14.  -20.]
 [-100.  -20.   50.]]
beta = inv(X'X) X'y = [ 2. -1.  4.]
LinearRegression intercept 2.000000000000 coef [-1.  4.]
max |difference| 8.62e-14
```

Every number in the hand calculation, to the last digit. The 8.62×10^{-14} is floating point, not disagreement. Note that sklearn does *not* form the inverse — it calls `scipy.linalg.lstsq`, which is slower to describe and far more numerically stable.

A shortcut worth knowing: centre first

Subtract the column means. The intercept then separates out and you invert a 2×2 instead of a 3×3 :

$$\begin{pmatrix} S_{11} & S_{12} \\ S_{12} & S_{22} \end{pmatrix} \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} = \begin{pmatrix} S_{1y} \\ S_{2y} \end{pmatrix}, \quad b_0 = \bar{y} - b_1 \bar{x}_1 - b_2 \bar{x}_2.$$

```
centred cross-product matrix
[[10.  4. ]
 [ 4.  2.8]]
det 12.0  inverse * 12 =
[[ 2.8 -4. ]
 [-4.  10. ]]
X'y centred      [6.0 7.2]
slopes from the 2x2 system [-1.  4.]
intercept = ybar - b1*x1bar - b2*x2bar = 2.0
```

A shortcut worth knowing: centre first

Subtract the column means. The intercept then separates out and you invert a 2×2 instead of a 3×3 :

$$\begin{pmatrix} S_{11} & S_{12} \\ S_{12} & S_{22} \end{pmatrix} \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} = \begin{pmatrix} S_{1y} \\ S_{2y} \end{pmatrix}, \quad b_0 = \bar{y} - b_1 \bar{x}_1 - b_2 \bar{x}_2.$$

```
centred cross-product matrix
[[10.  4. ]
 [ 4.  2.8]]
det 12.0  inverse * 12 =
[[ 2.8 -4. ]
 [-4.  10. ]]
X'y centred      [6.0 7.2]
slopes from the 2x2 system [-1.  4.]
intercept = ybar - b1*x1bar - b2*x2bar = 2.0
```

$$b_1 = \frac{2.8(6.0) - 4(7.2)}{12} = \frac{-12}{12} = -1, \quad b_2 = \frac{-4(6.0) + 10(7.2)}{12} = \frac{48}{12} = 4. \text{ Two lines of arithmetic and the same answer. This is the form to use in an examination.}$$

Drawn: the marginal picture and the fitted plane



Left: ignore area and bedrooms look good — the orange line has slope $+0.6$. But join the two flats that share a floor area and the dashed segments go down. Right: the fitted plane, with the five residuals drawn as vertical bars summing to $SSE = 4$.

“Holding the others fixed”

b_j is the change in $\hat{y}$ for a one-unit increase in x_j **when every other predictor in the model is held constant.**

Three words that do all the work

In the model. Not “in the world”. The clause names the other columns *you happened to include*. Add a column, remove a column, and b_j changes — legitimately, because it is now answering a different question.

The flats again

$b_1 = -1$ does not say bedrooms make a flat cheaper. It says: among flats of the *same area*, the one chopped into more bedrooms is worth less. Both statements can be true, and the data contain both.

And the clause can be physically impossible

“Holding total cholesterol fixed, LDL cholesterol has effect b ” describes an intervention nobody can perform. The number is still computable, and still means something — but not what a headline would say it means.

Predict first #1

`load_diabetes(scaled=False)`: 442 patients, 10 measurements in their original units, target is disease progression after one year.

Regress the target on `s1`, total serum cholesterol, **on its own**. The slope is **+0.4723** per mg/dl — more cholesterol, worse progression, as everyone expects.

Now put the other nine measurements into the model. What happens to the `s1` coefficient?

Predict first #1

`load_diabetes(scaled=False)`: 442 patients, 10 measurements in their original units, target is disease progression after one year.

Regress the target on `s1`, total serum cholesterol, **on its own**. The slope is **+0.4723** per mg/dl — more cholesterol, worse progression, as everyone expects.

Now put the other nine measurements into the model. What happens to the `s1` coefficient?

```
s1 = total serum cholesterol: +0.4723 alone, -1.0900 in the full model  
ratio of magnitudes 2.31, and the sign is reversed
```

Predict first #1

`load_diabetes(scaled=False)`: 442 patients, 10 measurements in their original units, target is disease progression after one year.

Regress the target on `s1`, total serum cholesterol, **on its own**. The slope is **+0.4723** per mg/dl — more cholesterol, worse progression, as everyone expects.

Now put the other nine measurements into the model. What happens to the `s1` coefficient?

```
s1 = total serum cholesterol: +0.4723 alone, -1.0900 in the full model  
ratio of magnitudes 2.31, and the sign is reversed
```

Positive on its own, negative in company — and bigger

Not “smaller”. Not “about the same”. **The opposite sign and 2.3 times the magnitude.** Nothing was done to the column; nine other columns joined the model. Both numbers are correct answers to different questions.

And it is not a one-off: four of the ten flip

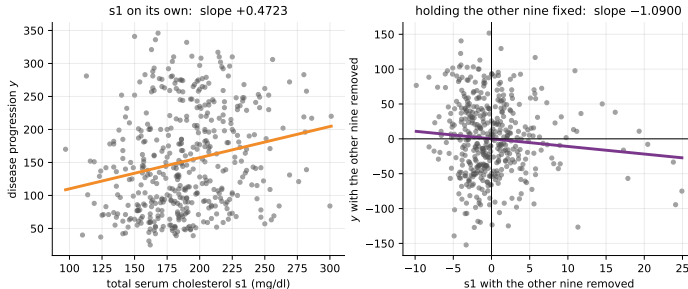
feature	simple slope	slope in the full 10-feature model	
age	+1.1050	-0.0364	<-- SIGN FLIP
sex	+6.6454	-22.8596	<-- SIGN FLIP
bmi	+10.2331	+5.6030	
bp	+2.4607	+1.1168	
s1	+0.4723	-1.0900	<-- SIGN FLIP
s2	+0.4412	+0.7465	
s3	-2.3531	+0.3720	<-- SIGN FLIP
s4	+25.7158	+6.5338	
s5	+83.5114	+68.4831	
s6	+2.5649	+0.2801	

And it is not a one-off: four of the ten flip

feature	simple slope	slope in the full 10-feature model	
age	+1.1050	-0.0364	<-- SIGN FLIP
sex	+6.6454	-22.8596	<-- SIGN FLIP
bmi	+10.2331	+5.6030	
bp	+2.4607	+1.1168	
s1	+0.4723	-1.0900	<-- SIGN FLIP
s2	+0.4412	+0.7465	
s3	-2.3531	+0.3720	<-- SIGN FLIP
s4	+25.7158	+6.5338	
s5	+83.5114	+68.4831	
s6	+2.5649	+0.2801	

Ten simple regressions are not one multiple regression. Every simple slope absorbs the effect of everything correlated with that column; every multiple slope has had those effects removed. Reporting the first and calling it the second is the commonest mistake in applied regression.

Drawn: the same column, two answers



Left: raw s1 against the target — an upward cloud, slope +0.4723. Right: the **added-variable plot**. Both axes have had the other nine predictors regressed out, and what is left slopes *down* at -1.0900. Same 442 patients, same column, opposite conclusion.

Where the difference went

Fit y on x_1 alone, and y on x_1 and x_2 . Then $b_1^{\text{simple}} = b_1 + b_2 \delta$, where δ is the slope of x_2 regressed on x_1 .

```
s1 (total cholesterol) alone           +0.4723
s1 with s5 (log triglycerides) added    -0.2418
s5 in that pair                         +91.7683
corr(s1, s5) +0.5155
  regress s5 on s1: delta = +0.007781
-0.2418 + +91.7683 * +0.007781 = +0.4723   (simple slope +0.4723)
```

Where the difference went

Fit y on x_1 alone, and y on x_1 and x_2 . Then $b_1^{\text{simple}} = b_1 + b_2 \delta$, where δ is the slope of x_2 regressed on x_1 .

```
s1 (total cholesterol) alone          +0.4723
s1 with s5 (log triglycerides) added  -0.2418
s5 in that pair                       +91.7683
corr(s1, s5) +0.5155
  regress s5 on s1: delta = +0.007781
-0.2418 + +91.7683 * +0.007781 = +0.4723   (simple slope +0.4723)
```

The simple slope was $91.77 \times 0.0078 = 0.714$ of borrowed effect

s1 was carrying s5's effect on its back, because the two correlate at +0.52. Remove the loan and what is left is -0.24. **Two predictors are enough to flip the sign; the other eight take it to -1.09.**

Partial regression coefficients: the recipe

The **partial regression coefficient** of x_j is what you get from a two-step construction that never mentions the other coefficients:

1. Regress x_j on all the *other* predictors; keep the residual $\tilde{x}_j$ — the part of x_j nothing else explains.
2. Regress y on all the other predictors; keep the residual $\tilde{y}$.
3. Regress $\tilde{y}$ on $\tilde{x}_j$. That single slope is b_j .

```
residual of s1 after regressing it on the other nine    sd 4.4979
residual of y  after regressing it on the other nine    sd 53.7607
slope of (y residual) on (s1 residual)  -1.0899963341
coefficient of s1 in the full model      -1.0899963341
difference 2.15e-14
```

Partial regression coefficients: the recipe

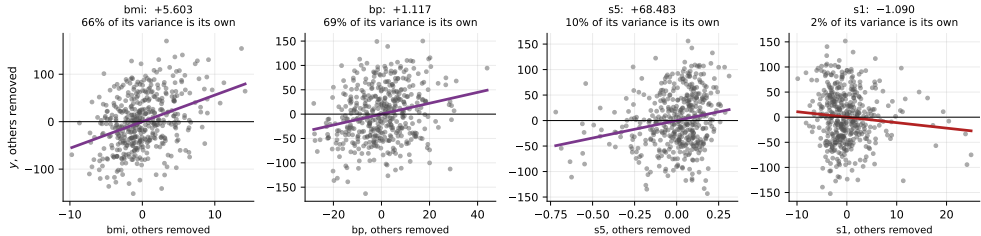
The **partial regression coefficient** of x_j is what you get from a two-step construction that never mentions the other coefficients:

1. Regress x_j on all the *other* predictors; keep the residual $\tilde{x}_j$ — the part of x_j nothing else explains.
2. Regress y on all the other predictors; keep the residual $\tilde{y}$.
3. Regress $\tilde{y}$ on $\tilde{x}_j$. That single slope is b_j .

```
residual of s1 after regressing it on the other nine    sd 4.4979
residual of y  after regressing it on the other nine    sd 53.7607
slope of (y residual) on (s1 residual)  -1.0899963341
coefficient of s1 in the full model      -1.0899963341
difference 2.15e-14
```

This is the **Frisch–Waugh–Lovell** theorem, and it is the precise content of “holding the others fixed”. The scatter of $\tilde{y}$ against $\tilde{x}_j$ is the **added-variable plot** — the only honest two-dimensional picture of a multiple-regression coefficient.

Drawn: four added-variable plots



The slope of each cloud *is* the coefficient in the ten-predictor model. Read the second line of each title: 66% **of bmi's variance survives the other nine**, and **only 1.7% of s1's does**. That is why s1's cloud is narrow — and why its coefficient will turn out to be untrustworthy.

Raw coefficients cannot be compared

b_j has units of y per unit of x_j . Two coefficients with different denominators are **not the same kind of number**.

The fix

Standardise both sides before fitting, or equivalently rescale afterwards:

$$\beta_j^* = b_j \frac{s_j}{s_y}.$$

β_j^* is the change in y , **in standard deviations of y** , per one standard deviation of x_j .
Unit-free, and therefore comparable.

Where the formula comes from

Replace x_j by $(x_j - \bar{x}_j)/s_j$ and y by $(y - \bar{y})/s_y$. The design matrix column is divided by s_j , so its coefficient is multiplied by s_j ; the target is divided by s_y , so every coefficient is divided by s_y .

One thing it does not fix

β^* is still a *conditional* quantity.

Predict first #2

The full ten-predictor diabetes model, in original units. The largest raw coefficient by a wide margin is s5, log serum triglycerides, at **+68.48**; s1 is a mere -1.09 , sixth of ten. Their standard deviations: s5 has $s_j = 0.52$ and s1 has $s_j = 34.61$.

Which predictor has the largest standardized coefficient, and where does s1 land?

Predict first #2

The full ten-predictor diabetes model, in original units. The largest raw coefficient by a wide margin is s5, log serum triglycerides, at **+68.48**; s1 is a mere -1.09 , sixth of ten. Their standard deviations: s5 has $s_j = 0.52$ and s1 has $s_j = 34.61$.

Which predictor has the largest standardized coefficient, and where does s1 land?

feature	raw coef	sd	beta*	rank raw	rank beta*
s1	-1.0900	34.6081	-0.4893	6	1
s5	+68.4831	0.5224	+0.4640	1	2
bmi	+5.6030	4.4181	+0.3211	4	3
sex	-22.8596	0.4996	-0.1481	2	6

Predict first #2

The full ten-predictor diabetes model, in original units. The largest raw coefficient by a wide margin is s5, log serum triglycerides, at **+68.48**; s1 is a mere -1.09 , sixth of ten. Their standard deviations: s5 has $s_j = 0.52$ and s1 has $s_j = 34.61$.

Which predictor has the largest standardized coefficient, and where does s1 land?

feature	raw coef	sd	beta*	rank raw	rank beta*
s1	-1.0900	34.6081	-0.4893	6	1
s5	+68.4831	0.5224	+0.4640	1	2
bmi	+5.6030	4.4181	+0.3211	4	3
sex	-22.8596	0.4996	-0.1481	2	6

Sixth becomes first; second becomes sixth

s1 moves from rank 6 to rank **1** and sex from 2 to 6. **The raw ranking was a ranking of measurement units:** s5 is a logarithm and spans half a unit, s1 is in mg/dl and spans a hundred.

And β^* really is unit-free — check it

Record bmi in units of 10^{-4} kg/m² instead. Same data, same model, same predictions.

bmi coefficient	+5.6030	(kg/m2)	+0.00056030	(1e-4 kg/m2)
beta* for bmi	+0.321100	and	+0.321100	-- unchanged

And β^* really is unit-free — check it

Record bmi in units of 10^{-4} kg/m^2 instead. Same data, same model, same predictions.

bmi coefficient	+5.6030	(kg/m2)	+0.00056030	(1e-4 kg/m2)
-----------------	---------	---------	-------------	--------------

beta* for bmi	+0.321100	and	+0.321100	-- unchanged
---------------	-----------	-----	-----------	--------------

check against a regression on standardised columns: max diff 7.72e-15

And β^* really is unit-free — check it

Record bmi in units of 10^{-4} kg/m² instead. Same data, same model, same predictions.

```
bmi coefficient +5.6030 (kg/m2) +0.00056030 (1e-4 kg/m2)
beta* for bmi +0.321100 and +0.321100 -- unchanged
```

```
check against a regression on standardised columns: max diff 7.72e-15
```

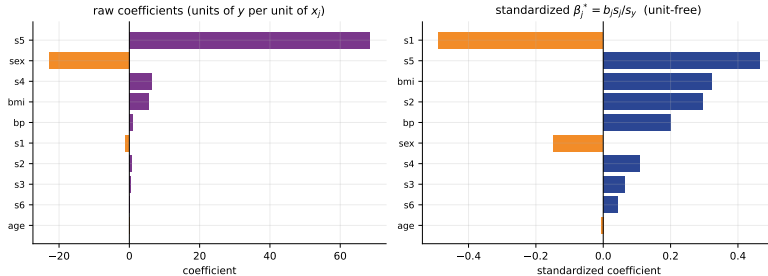
Two routes, one answer

Rescaling b_j by s_j/s_y gives *exactly* what you get by putting `StandardScaler` in front of the regression. Use whichever is convenient.

Do not over-read β^* either

A large β^* can still come from a collinear column with a wild coefficient, and s_j depends on *this* sample. It ranks; it does not adjudicate.

Drawn: two rankings of the same model



Left: raw coefficients, sorted. One bar is fifty times the others and it tells you nothing except that s5 is measured in small numbers. Right: the same fit, standardised. **Five predictors now sit within a factor of two of each other, and s1 has come first.**

Near-dependence: the matrix is invertible, barely

The quantity that matters

Let R_j^2 be the R^2 from regressing x_j on *all the other predictors*. Then

$$\text{VIF}_j = \frac{1}{1 - R_j^2}, \quad \text{Var}(\hat{\beta}_j) = \frac{\sigma^2}{(n-1)s_j^2} \cdot \text{VIF}_j.$$

The **variance inflation factor** is the factor by which collinearity multiplies the variance of $\hat{\beta}_j$. Standard errors are multiplied by $\sqrt{\text{VIF}_j}$.

Reading it

$R_j^2 = 0 \Rightarrow \text{VIF} = 1$: no inflation at all.

$R_j^2 = 0.9 \Rightarrow \text{VIF} = 10$.

$R_j^2 = 0.99 \Rightarrow \text{VIF} = 100$, so ten times the

sta

Rules of thumb, and their status

VIF > 5 deserves a look, > 10 is usually a problem. They are conventions, not theorems — what matters is whether the coefficient you

VIF on the diabetes table

```
def vif(A):  
    return np.array([1 / (1 - LinearRegression()  
        .fit(np.delete(A, j, 1), A[:, j])  
        .score(np.delete(A, j, 1), A[:, j])) for j in range(A.shape[1])])
```

feature	R2_j	VIF	sqrt(VIF)
s1	0.9831	59.20	7.69
s2	0.9745	39.19	6.26
s3	0.9351	15.40	3.92
s5	0.9008	10.08	3.17
s4	0.8875	8.89	2.98
bmi	0.3375	1.51	1.23
s6	0.3264	1.48	1.22
bp	0.3148	1.46	1.21
sex	0.2176	1.28	1.13
age	0.1785	1.22	1.10

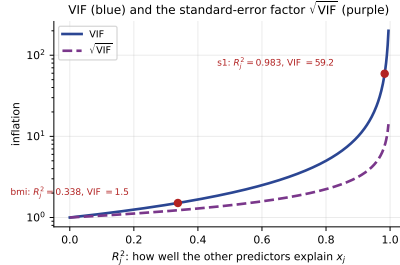
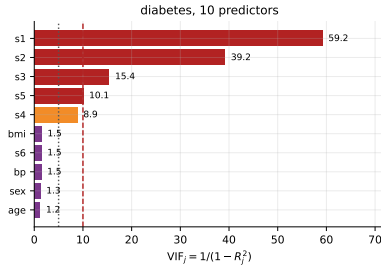
VIF on the diabetes table

```
def vif(A):  
    return np.array([1 / (1 - LinearRegression()  
        .fit(np.delete(A, j, 1), A[:, j])  
        .score(np.delete(A, j, 1), A[:, j])) for j in range(A.shape[1])])
```

feature	R2_j	VIF	sqrt(VIF)
s1	0.9831	59.20	7.69
s2	0.9745	39.19	6.26
s3	0.9351	15.40	3.92
s5	0.9008	10.08	3.17
s4	0.8875	8.89	2.98
bmi	0.3375	1.51	1.23
s6	0.3264	1.48	1.22
bp	0.3148	1.46	1.21
sex	0.2176	1.28	1.13
age	0.1785	1.22	1.10

The five serum measurements are five views of one blood sample: $s4 = s1/s3$ by definition, and $s2$ is a component of $s1$. **98.3% of $s1$ is predictable from the other nine columns**, so its standard error is 7.7 times what it would be if the columns were orthogonal.

Drawn: variance inflation, measured and plotted



Left: the ten VIFs, with the conventional thresholds at 5 and 10. Right: the curve $1/(1 - R_j^2)$ on a log axis. **It is flat until $R_j^2 \approx 0.8$ and then vertical** — which is why collinearity tends to arrive suddenly rather than gradually.

Predict first #3

Resample the 442 patients with replacement, refit, and repeat 2000 times. Every resample is a plausible alternative dataset from the same population.

The fitted coefficient of `s1` is -1.090 , and of `bmi` is $+5.603$.

How wide are the two bootstrap distributions, and does either of them cross zero?

Predict first #3

Resample the 442 patients with replacement, refit, and repeat 2000 times. Every resample is a plausible alternative dataset from the same population.

The fitted coefficient of `s1` is -1.090 , and of `bmi` is $+5.603$.

How wide are the two bootstrap distributions, and does either of them cross zero?

feature	coef	boot sd	2.5%	97.5%	VIF	sign flips
s1	-1.090	0.565	-2.093	+0.113	59.2	3.8%
s2	+0.746	0.512	-0.325	+1.634	39.2	8.6%
s5	+68.483	14.982	+39.101	+97.225	10.1	0.0%
bmi	+5.603	0.721	+4.159	+6.996	1.5	0.0%

Predict first #3

Resample the 442 patients with replacement, refit, and repeat 2000 times. Every resample is a plausible alternative dataset from the same population.

The fitted coefficient of `s1` is -1.090 , and of `bmi` is $+5.603$.

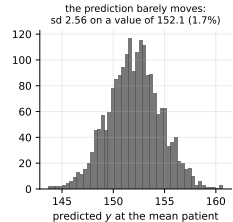
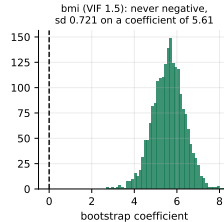
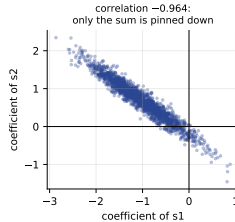
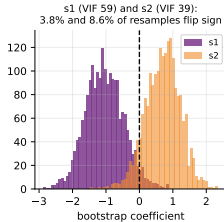
How wide are the two bootstrap distributions, and does either of them cross zero?

feature	coef	boot sd	2.5%	97.5%	VIF	sign flips
<code>s1</code>	-1.090	0.565	-2.093	$+0.113$	59.2	3.8%
<code>s2</code>	$+0.746$	0.512	-0.325	$+1.634$	39.2	8.6%
<code>s5</code>	$+68.483$	14.982	$+39.101$	$+97.225$	10.1	0.0%
<code>bmi</code>	$+5.603$	0.721	$+4.159$	$+6.996$	1.5	0.0%

`s2` changes sign in one resample out of twelve

`bmi`'s interval is $[+4.16, +7.00]$ — never near zero, VIF 1.5. `s1`'s is $[-2.10, +0.06]$ and `s2`'s is $[-0.30, +1.69]$: **both straddle zero**, and neither sign is established by 442 patients. **The correlation between the bootstrap `s1` and `s2` coefficients is -0.964 .**

Drawn: what the bootstrap sees



Panels 1 and 2: the s1 and s2 coefficients slide along a line of slope -1 — **their sum is pinned down and the split between them is not**. Panel 3: bmi, VIF 1.5, never negative. Panel 4: **the prediction at the mean patient has a spread of 1.7%**, while the coefficients feeding it are barely determined.

Predicting is fine. Explaining is not.

Two predictors correlated at 0.9986; the truth is $y = 3x_1 + 2x_2 + \varepsilon$. Fit on the first hundred rows, then on the second hundred.

```
corr(a, b) 0.9986      true coefficients 3.0 and 2.0      VIF [351.6 351.6]
first half  a  +2.553  b  +2.460  sum +5.012  test R2 +0.9895
second half a  +2.208  b  +2.848  sum +5.056  test R2 +0.9895
the two coefficient vectors differ by 0.39, their sum by 0.044
```

Predicting is fine. Explaining is not.

Two predictors correlated at 0.9986; the truth is $y = 3x_1 + 2x_2 + \varepsilon$. Fit on the first hundred rows, then on the second hundred.

```
corr(a, b) 0.9986      true coefficients 3.0 and 2.0      VIF [351.6 351.6]
first half  a  +2.553  b  +2.460  sum  +5.012  test R2 +0.9895
second half a  +2.208  b  +2.848  sum  +5.056  test R2 +0.9895
the two coefficient vectors differ by 0.39, their sum by 0.044

cross-validated R2, all 10 features      +0.4892
cross-validated R2, s1 dropped           +0.4867
cross-validated R2, s1 and s2 both dropped +0.4867
```

Predicting is fine. Explaining is not.

Two predictors correlated at 0.9986; the truth is $y = 3x_1 + 2x_2 + \varepsilon$. Fit on the first hundred rows, then on the second hundred.

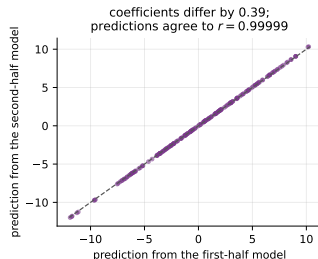
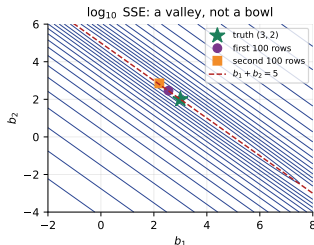
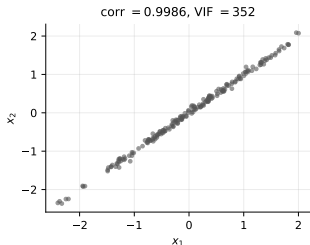
```
corr(a, b) 0.9986      true coefficients 3.0 and 2.0      VIF [351.6 351.6]
first half  a  +2.553  b  +2.460  sum +5.012  test R2 +0.9895
second half a  +2.208  b  +2.848  sum +5.056  test R2 +0.9895
the two coefficient vectors differ by 0.39, their sum by 0.044

cross-validated R2, all 10 features      +0.4892
cross-validated R2, s1 dropped           +0.4867
cross-validated R2, s1 and s2 both dropped +0.4867
```

One model says $b_2 = +1.91$, the other says -0.60

Both score $R^2 = 0.989$ on the same test data. Dropping the worst two diabetes predictors costs 0.0025 of cross-validated R^2 . **Multicollinearity is a disease of the coefficients, not of the predictions.** If you only need $\hat{y}$, you may ignore it. If anyone will read a coefficient, you may not.

Drawn: a valley, not a bowl



Middle panel: the SSE contours are a long trench along $b_1 + b_2 = 5$. Any point in the trench fits almost equally well, and which one you land on is decided by noise — purple and orange are two halves of the same dataset. Right: **their predictions agree to $r = 0.99999$.**

What to do about collinearity

	What it does	Cost
Nothing	fine if you only predict	no coefficient may be interpreted
Drop a column	removes the redundancy	the survivor now carries both effects; the choice is scientific, not statistical
Combine columns	a sum, a ratio, an index	you must be able to name the combination
Ridge / lasso	$X^T X + \lambda I$ is always invertible	biased coefficients, chosen by cross-validation — <i>next lecture</i>
Collect more data	shrinks every standard error	does not help if the columns are collinear <i>by definition</i>

Measured, on diabetes

Least squares on standardised columns gives $s_1 = -37.68$ and $s_2 = +22.68$; ridge at $\alpha = 1$ gives -30.09 and $+16.65$, and cross-validated R^2 moves from 0.4892 to 0.4896. **Same predictions, calmer coefficients.**

Why missing data is worse in multiple regression

Least squares needs a **complete row**. One missing cell in one column removes the whole patient.

The arithmetic of complete cases

With d columns each missing independently at rate p , a row survives with probability $(1 - p)^d$.

p	rows kept of 309	%
0.01	279	90.4
0.05	185	59.9
0.10	108	34.9

Carried over from Lecture 5

MCAR — complete cases are a fair sample; you lose power, not correctness. **MAR** — complete cases are biased, but the observed columns can undo it. **MNAR** — the missingness depends on the value itself and no amount of arithmetic repairs it.

The multiple-regression twist

Measured: five ways to handle it

309 training rows, 10 predictors, MCAR missingness, evaluated on a held-out third that is complete.

```
no missingness at all          test R2 +0.3929

MCAR at 10%: cells missing 10.4%, complete rows 103 of 309 (33.3%)
listwise deletion              103 rows   test R2 +0.3231
mean imputation                309 rows   test R2 +0.3875
mean + missing indicators       309 rows   test R2 +0.3657
IterativeImputer (MICE)        309 rows   test R2 +0.3871
```

Measured: five ways to handle it

309 training rows, 10 predictors, MCAR missingness, evaluated on a held-out third that is complete.

no missingness at all		test R2 +0.3929
MCAR at 10%: cells missing 10.4%, complete rows 103 of 309 (33.3%)		
listwise deletion	103 rows	test R2 +0.3231
mean imputation	309 rows	test R2 +0.3875
mean + missing indicators	309 rows	test R2 +0.3657
IterativeImputer (MICE)	309 rows	test R2 +0.3871
MCAR at 25%: cells missing 25.2%, complete rows 13 of 309 (4.2%)		
listwise deletion	13 rows	test R2 -2.1451
mean imputation	309 rows	test R2 +0.4048
IterativeImputer (MICE)	309 rows	test R2 +0.4038

Measured: five ways to handle it

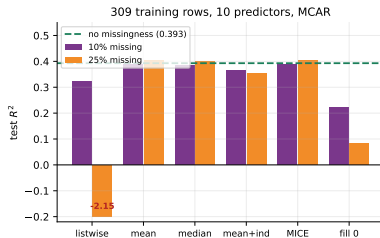
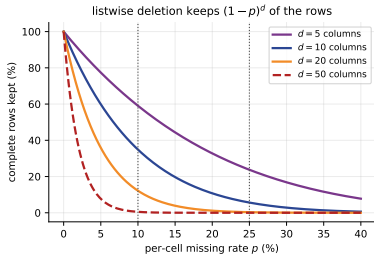
309 training rows, 10 predictors, MCAR missingness, evaluated on a held-out third that is complete.

no missingness at all		test R2 +0.3929
MCAR at 10%: cells missing 10.4%, complete rows 103 of 309 (33.3%)		
listwise deletion	103 rows	test R2 +0.3231
mean imputation	309 rows	test R2 +0.3875
mean + missing indicators	309 rows	test R2 +0.3657
IterativeImputer (MICE)	309 rows	test R2 +0.3871
MCAR at 25%: cells missing 25.2%, complete rows 13 of 309 (4.2%)		
listwise deletion	13 rows	test R2 -2.1451
mean imputation	309 rows	test R2 +0.4048
IterativeImputer (MICE)	309 rows	test R2 +0.4038

–2.15: worse than predicting the mean of y every time

At 25%, listwise deletion leaves 13 rows for 11 parameters and the model is nonsense. **Mean imputation, the crudest method there is, gives +0.405 on the same data**, and MICE +0.404, against a no-missingness +0.393. Those last two differ by 0.001: **do not rank them**. The measurable effect is that deleting rows costs 2.54 and imputing costs nothing.

Drawn: the cliff, and what climbs it



Left: $(1-p)^d$ for four widths of table. At $d=50$ a 5% cell rate keeps a fifth of your rows. Right: the five strategies at two rates. The listwise bar at 25% is off the bottom of the axis at -3.26 ; **filling with 0 is the other bad answer**, because a patient with age 0 and blood pressure 0 is a claim, not a gap.

The imputer is a learned step

```
# wrong: the column means are computed from every row, including test rows
X_filled = SimpleImputer().fit_transform(X)
cross_val_score(LinearRegression(), X_filled, y, cv=5)

# right: refitted on four fifths of the data inside every fold
cross_val_score(Pipeline([("imp", SimpleImputer()),
                           ("lr", LinearRegression())]), X, y, cv=5)
```

25% MCAR on all 442 rows, 5-fold cross-validation

mean	impute-then-split +0.4114	inside the Pipeline +0.4112	difference +0.0002
MICE	impute-then-split +0.4434	inside the Pipeline +0.4392	difference +0.0042

The imputer is a learned step

```
# wrong: the column means are computed from every row, including test rows
X_filled = SimpleImputer().fit_transform(X)
cross_val_score(LinearRegression(), X_filled, y, cv=5)
```

```
# right: refitted on four fifths of the data inside every fold
cross_val_score(Pipeline([("imp", SimpleImputer()),
                           ("lr", LinearRegression())]), X, y, cv=5)
```

25% MCAR on all 442 rows, 5-fold cross-validation

mean	impute-then-split +0.4114	inside the Pipeline +0.4112	difference +0.0002
MICE	impute-then-split +0.4434	inside the Pipeline +0.4392	difference +0.0042

Report this honestly: the leak here is tiny

+0.0002 and +0.0042. An imputer never looks at y , so there is very little for it to absorb — exactly the argument Lecture 6 made for PCA. **Put it in the Pipeline anyway:** it costs nothing, and a target-aware imputer on a small test set is a different story.

Three different questions, three different checks

Question	Check	What failure looks like
Are the <i>assumptions</i> met?	residual plots, Q–Q plot, scale–location	curvature, a fan, heavy tails
Does it <i>generalise</i> ?	train against test, or cross-validation	a large gap between the two
Is the extra column <i>earning its place</i> ?	adjusted R^2 , held-out R^2 , an F test	R^2 up, adjusted R^2 and test R^2 down

And remember what is guaranteed

$\sum e_i = 0$ and $\text{corr}(e, \hat{y}) = 0$ are forced by the normal equations. Measured on diabetes: residual mean -8.1×10^{-14} , correlation with fitted 5.4×10^{-16} . **Neither is evidence of anything.**

Residual diagnostics on the diabetes fit

```
diabetes, 10 features, 309 training rows  
train R2 +0.5539    test R2 +0.3929  
train RMSE 52.954  test RMSE 55.652  
Shapiro-Wilk on the residuals: W 0.9959, p 0.6100  
corr(|residual|, fitted) +0.2079, p 0.0002
```

Residual diagnostics on the diabetes fit

```
diabetes, 10 features, 309 training rows
train R2 +0.5539    test R2 +0.3929
train RMSE 52.954  test RMSE 55.652
Shapiro-Wilk on the residuals: W 0.9959, p 0.6100
corr(|residual|, fitted) +0.2079, p 0.0002
```

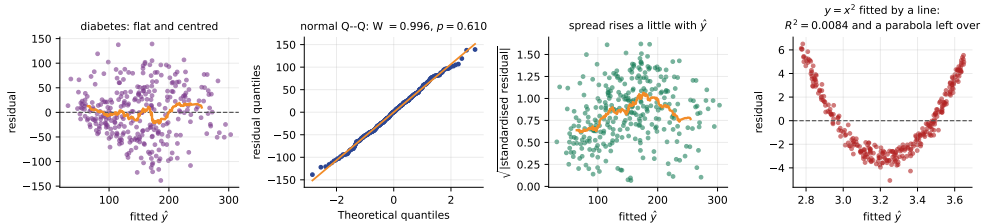
What passed

Normality: $p = 0.11$, no evidence against it.
Train and test RMSE agree to within about 5%,
so no overfitting worth worrying about at
 309×10 .

What did not

$|e|$ rises with $\hat{y}$ ($p = 0.0002$): mild
heteroscedasticity. The coefficients are still
unbiased; the standard errors are not, and
prediction intervals will be too narrow at the
top end.

Drawn: four diagnostic panels



The first three are the diabetes fit: flat, straight, slightly fanning. **The fourth is the point of the slide** — $y = x^2$ fitted by a straight line. $R^2 = 0.0000$, the residual mean is zero and the residual correlation with $\hat{y}$ is zero, and the residual plot is a parabola. **Look at the picture, not only at the summary.**

Predict first #4

80 training rows of diabetes, all 10 real predictors, train $R^2 = 0.5911$. Now append k columns of $\mathcal{N}(0, 1)$ noise, independent of everything, and refit.

What happens to the training R^2 ? To the adjusted R^2 ? To the R^2 on held-out rows?

Predict first #4

80 training rows of diabetes, all 10 real predictors, train $R^2 = 0.5911$. Now append k columns of $\mathcal{N}(0, 1)$ noise, independent of everything, and refit.

What happens to the training R^2 ? To the adjusted R^2 ? To the R^2 on held-out rows?

junk	k	p	train R2	adj R2	test R2
	0	10	0.5911	0.5318	0.2488
	1	11	0.5992	0.5344	0.2295
	5	15	0.6183	0.5289	0.1719
	10	20	0.6407	0.5188	0.1340
	20	30	0.6935	0.5059	-0.0794

Predict first #4

80 training rows of diabetes, all 10 real predictors, train $R^2 = 0.5911$. Now append k columns of $\mathcal{N}(0, 1)$ noise, independent of everything, and refit.

What happens to the training R^2 ? To the adjusted R^2 ? To the R^2 on held-out rows?

junk k	p	train R2	adj R2	test R2
0	10	0.5911	0.5318	0.2488
1	11	0.5992	0.5344	0.2295
5	15	0.6183	0.5289	0.1719
10	20	0.6407	0.5188	0.1340
20	30	0.6935	0.5059	-0.0794

R^2 rose at every one of the eight steps. It always does.

Adding a column can never increase SSE — the old solution is still available with a coefficient of zero — so R^2 is **monotone non-decreasing in p , whatever the column contains**. Adjusted R^2 was **fooled by the first junk column** and peaked there rather than at none, and test R^2 fell at **seven of eight steps**, from +0.249 to **-0.079** — past zero.

Adjusted R^2 : the correction and its limits

$$R^2 = 1 - \frac{\text{SSE}}{\text{SST}}, \quad R^2_{\text{adj}} = 1 - \frac{\text{SSE}/(n - p - 1)}{\text{SST}/(n - 1)} = 1 - (1 - R^2) \frac{n - 1}{n - p - 1}.$$

What the correction does

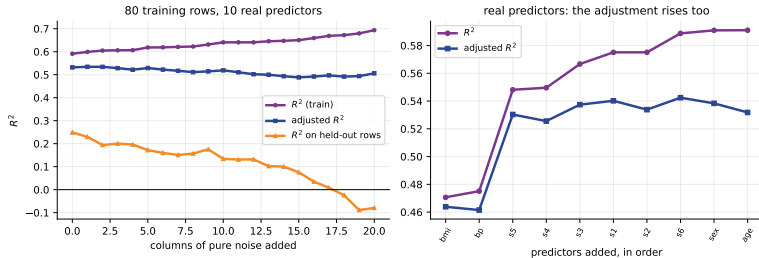
R^2 compares two sums; R^2_{adj} compares two *mean* squares. A useless column reduces SSE a little and the denominator $n - p - 1$ a lot, so the ratio rises and R^2_{adj} falls. **It can go negative.**

Where it stops being useful

As $p \rightarrow n - 1$ the factor $(n - 1)/(n - p - 1)$ explodes and the statistic becomes noise. Measured: at $k = 60$ junk columns with 80 rows, train $R^2 = 0.976$, test $R^2 = -7.35$, and R^2_{adj} reads a cheerful 0.786.

Adjusted R^2 is a warning light, not a model-selection criterion. Held-out error is the criterion. On the five flats: $R^2 = 0.8507$ against $R^2_{\text{adj}} = 0.7015$, because two parameters on five rows is a lot of parameters.

Drawn: the two curves separating



Left: purple (R^2) only ever rises; blue (adjusted) drifts down; orange (held-out) falls off a cliff.
Right: the same three-column arithmetic when the added predictors are *real* — both curves rise together, and the dip at s1 and s2 is the adjustment noticing that those two columns are nearly redundant.

The whole thing, as one object

```
pipe = Pipeline([
    ("imp", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()), # so the coefficients are beta*
    ("model", LinearRegression()),
])
cross_val_score(pipe, X, y, cv=KFold(5, shuffle=True, random_state=0))
```

```
impute -> standardise -> least squares, 5-fold on the 10%-missing table
fold R2 [0.2632 0.3567 0.5213 0.3747 0.5293]
mean +0.4090 sd 0.1143
for comparison, the same pipeline on the complete table: +0.4892
```

The whole thing, as one object

```
pipe = Pipeline([
    ("imp", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()), # so the coefficients are beta*
    ("model", LinearRegression()),
])
cross_val_score(pipe, X, y, cv=KFold(5, shuffle=True, random_state=0))
```

```
impute -> standardise -> least squares, 5-fold on the 10%-missing table
fold R2 [0.2632 0.3567 0.5213 0.3747 0.5293]
mean +0.4090 sd 0.1143
for comparison, the same pipeline on the complete table: +0.4892
```

The column medians and the column means and standard deviations are recomputed inside every fold. **And read the fold-to-fold spread: 0.28 to 0.49.** A single train/test split of 442 patients would have reported any number in that range with equal confidence.

Summary — twelve lines

1. $y = X\beta + \varepsilon$. Differentiating $(y - Xb)^T(y - Xb)$ gives $\nabla S = -2X^T y + 2X^T Xb$, so $X^T X\hat{\beta} = X^T y$ and $\hat{\beta} = (X^T X)^{-1}X^T y$. The Hessian $2X^T X$ is positive semi-definite, so the stationary point is a global minimum.
2. $X^T X$ is invertible exactly when X has full column rank, because $\text{null}(X^T X) = \text{null}(X)$. Perfect collinearity and $p + 1 > n$ are the two ways to lose it.
3. Five flats, integer arithmetic: $\det X^T X = 60$, $\hat{\beta} = (2, -1, 4)$, residuals $(-1, 1, 0, 1, -1)$, $X^T e = 0$, $R^2 = 0.8507$, $R^2_{\text{adj}} = 0.7015$. Identical to `LinearRegression` at 8.6×10^{-14} .
4. b_j means “holding the others fixed”. The bedroom coefficient is -1 because flats B and D have the same area.
5. **s1 has slope +0.4723 alone and -1.0900 with nine others in the model**, and four of the ten diabetes predictors change sign that way.
6. The omitted-variable identity accounts for it exactly: $-0.2418 + 91.7683 \times 0.007781 = +0.4723$.
7. Frisch–Waugh–Lovell: residualise both sides on the other predictors and the single slope is b_j , to 2×10^{-14} . That scatter is the added-variable plot.
8. Standardizing changes the ranking: **s1 moves from sixth by raw magnitude to first by β^*** , and β^* is unchanged when bmi is rescaled by 10^4 .
9. $\text{VIF}_j = 1/(1 - R_j^2)$. Four of ten exceed 10; **s1 reaches 59.2**, so its standard error is $7.7\times$ inflated.
10. 2000 bootstrap resamples: **s1 and s2 straddle zero and correlate at -0.964** , while the prediction at the mean patient has a spread of 1.7%. On a collinear pair whose truth was $(3, 2)$, two halves gave $(+2.55, +2.46)$ and $(+2.21, +2.85)$ — both wrong — with test $R^2 = 0.9895$ both times. **Collinearity breaks explanation, not prediction.**
11. 10% missing cells across 10 columns deletes two thirds of the rows; at 25% listwise deletion scores **-2.15** while mean imputation scores $+0.405$ and MICE $+0.404$.
12. R^2 rose at all eight steps as noise columns were added; adjusted R^2 was fooled by the first one, and held-out R^2 fell from $+0.249$ to -0.079 . **Only the held-out number is a criterion.**

Before the next lecture

Read

notes.pdf — the full matrix derivation with every step, the Frisch–Waugh–Lovell proof, the variance formula behind VIF, and **10 practice questions with worked solutions**.

Do

Questions 2, 5 and 9. Question 2 is a 3×3 inverse and $\hat{\beta}$ by hand; question 9 is a report you will be asked to review for real.

Next: ridge and lasso

Regularisation replaces $X^T X$ by $X^T X + \lambda I$, which is invertible for every $\lambda > 0$. **Every failure in this lecture is a reason that lecture exists.**

A habit to start now

Before you interpret any coefficient, print the VIFs and bootstrap the fit. If the interval straddles zero, say so — do not report a sign the data has not established.

Materials: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: James et al., *An Introduction to Statistical Learning with Python*, Ch. 3–6.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 3.; Deisenroth, Faisal and Ong, *Mathematics for Machine Learning*, Ch. 9.; Müller and Guido, *Introduction to Machine Learning with Python*, §2.3.